

Labidus: Streaming Async Instruction RISC-V Overlay

Labidus: RISC-V Overlay with Streaming Asynchronous Custom
Instructions

Gongjin Sun, Seongyoung Kang, Jane He, Se-Min Lim, and Sang-Woo Jun (2025)

Software-defined FPGA overlay에서 긴 operator latency와 범용 control overhead를 함께
줄이는 구조

FPGA 개발의 두 비용

Custom hardware와 processor overlay는 서로 다른 비용을 줄인다

접근	얻는 것	남는 비용
RTL	응용에 맞춘 pipeline과 data path	낮은 수준의 hardware design expertise
HLS	C 계열 언어에서 hardware 생성	비정규 loop와 control에 상세 guidance 필요
Soft processor overlay	기존 language와 compiler 활용	instruction decode와 control logic의 LUT overhead

Labidus는 overlay의 개발 편의성을 유지하면서 control overhead를 줄이는 문제를 다룬다.

Labidus의 설계 목표

- | | | |
|----|------------------|--------------------------------|
| 01 | C kernel | Programmer가 software로 계산을 표현 |
| ↓ | | |
| 02 | Static analysis | 반복되는 operation tree를 분석 |
| ↓ | | |
| 03 | Custom operators | 복잡한 연산을 fused instruction으로 구성 |
| ↓ | | |
| 04 | High utilization | Stream과 async queue로 실행을 겹침 |

핵심은 custom operator를 만드는 일보다 계속 바쁘게 유지하는 데 있다.

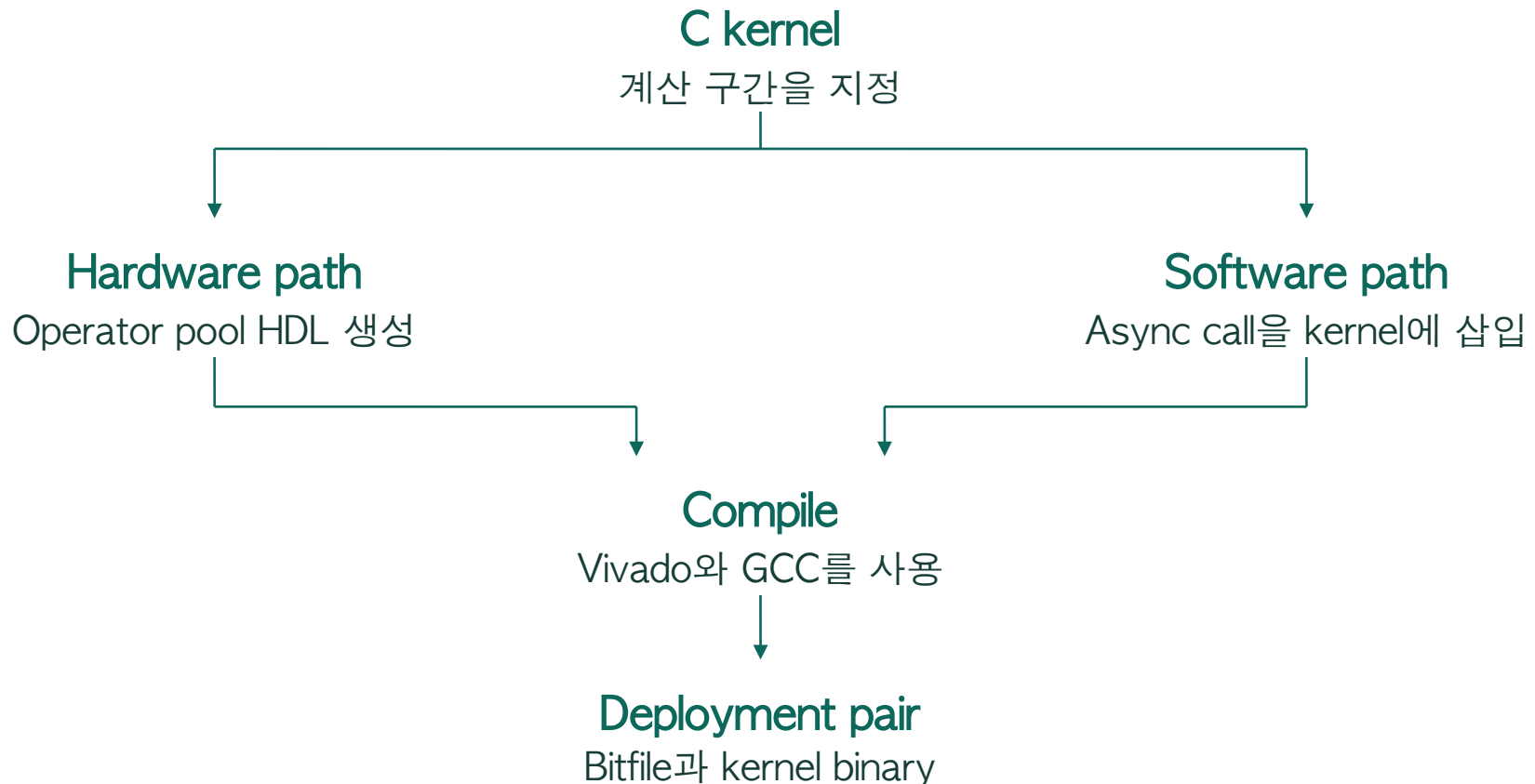
Programming model의 시스템 경계

영역	Labidus가 처리하는 범위	Programmer 책임
Host software	Generated accelerator를 호출	Host와 kernel의 interaction 작성
Kernel software	Custom instruction call이 삽입됨	C code와 compute-heavy region 제공
FPGA hardware	Operator pool과 core cluster 생성	현재 library에서는 offload region 지정

현재 Labidus는 host-kernel 연결과 offload 구간을 완전히 자동화하지 않는다.

C kernel에서 두 산출물까지

Static analysis는 hardware와 software를 함께 바꾼다



Haversine loop의 fused operator 생성

Operation tree를 leaf부터 묶되 two inputs와 one output 형태를 유지한다



Fused operator는 수십 cycle latency를 가질 수 있어도 매 cycle 새 input을 받을 수 있다.

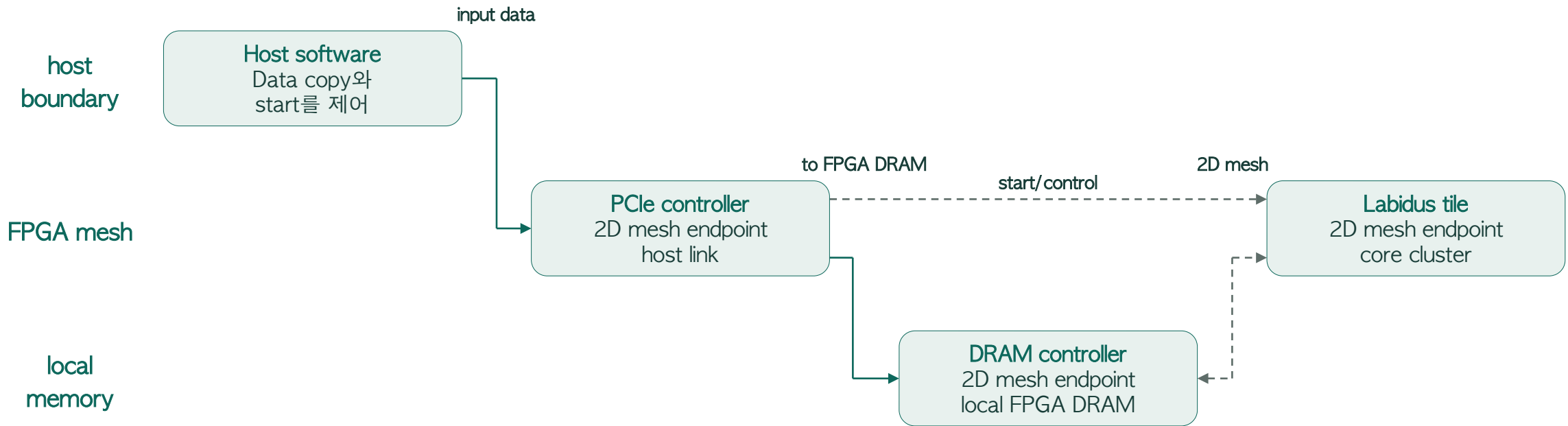
FPGA 배치와 실행 lifecycle

01	Configure FPGA	Generated HDL bitfile을 load
↓		
02	Load kernels	Compiled binary를 모든 core에 배치
↓		
03	Copy input	Host가 PCIe로 FPGA DRAM에 전송
↓		
04	Start cores	모든 core에 start signal 전달
↓		
05	Return results	FPGA DRAM에서 host로 복사

Evaluation 대상은 host와 PCIe로 연결된 FPGA accelerator card다.

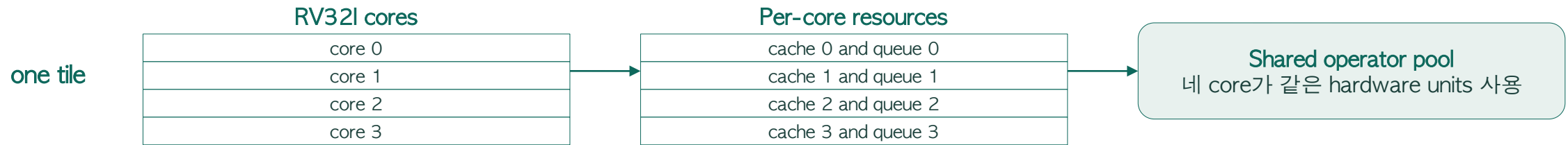
Host, memory, Labidus tile의 연결

실선은 host input을 DRAM까지, 점선은 start/control과 fabric connectivity를 보인다



Tile 내부 ring과 tile 간 2D mesh가 서로 다른 network level을 이룬다.

4-core tile과 shared operator pool



Operator를 core마다 복제하지 않고 공유해 낮은 utilization의 instance를 줄인다.

Core 안의 data path와 control path

구성요소

주요 역할

연결되는 대상

RV32I core

Custom op와 memory command issue

Completion queue와 local mem

Completion queue

Slot reservation과 async result 정렬

Operator pool과 stream cache

Stream cache

Off-chip burst를 stream으로 공급

Router, DRAM, completion queue

Operator pool

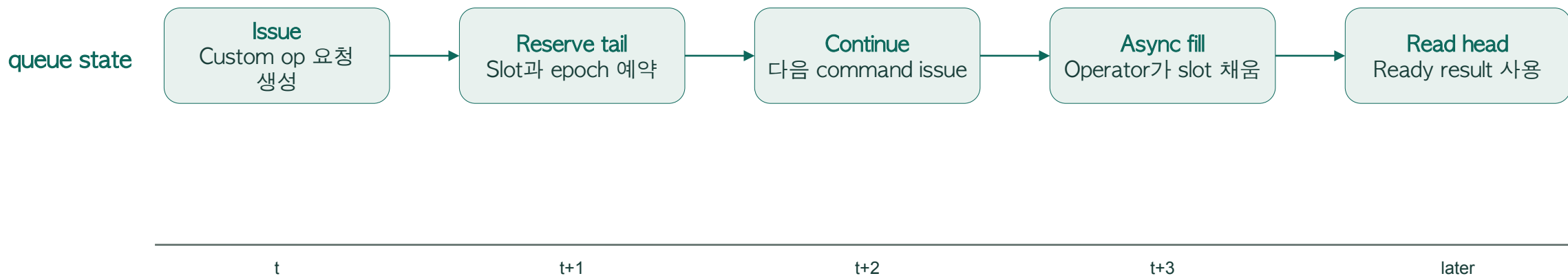
Fused computation을 pipeline 실행

Tile의 four cores가 공유

RV32I core는 control을 issue하고, queue와 cache가 data readiness를 흡수한다.

Asynchronous completion queue의 기본 동작

Operation issue 순서와 result completion 시점을 분리한다



Result가 head에 도달하기 전에 ready되면 operator latency를 다른 work와 겹칠 수 있다.

Circular buffer와 epoch tracking

01	Reserve slot	Tail 위치를 request에 연결
↓		
02	Tag epoch	현재 wrap generation을 request에 기록
↓		
03	Tail wraps	Buffer 끝에서 global epoch 증가
↓		
04	Response arrives	Address와 epoch가 slot을 식별
↓		
05	Check readiness	Old epoch slot과 new result를 구분

Epoch는 같은 physical slot을 여러 바퀴 재사용할 때 completion generation을 구분한다.

Latency hiding이 성립하는 조건

조건

Operator pipeline

In-flight capacity

Head readiness

Input stream

성립할 때

긴 latency에도 매 cycle input 수용

Queue가 여러 request를 유지

Head 도달 전에 result complete

Cache와 repeat가 새 operand 공급

깨질 때

Initiation interval이 길면 공급률 감소

Queue가 차면 새 request가 대기

Incomplete head read에서 stall

같은 register value만 반복

Labidus는 single-operation latency보다 pipeline utilization을 개선하는 구조다.

교육용 M=3 streaming trace

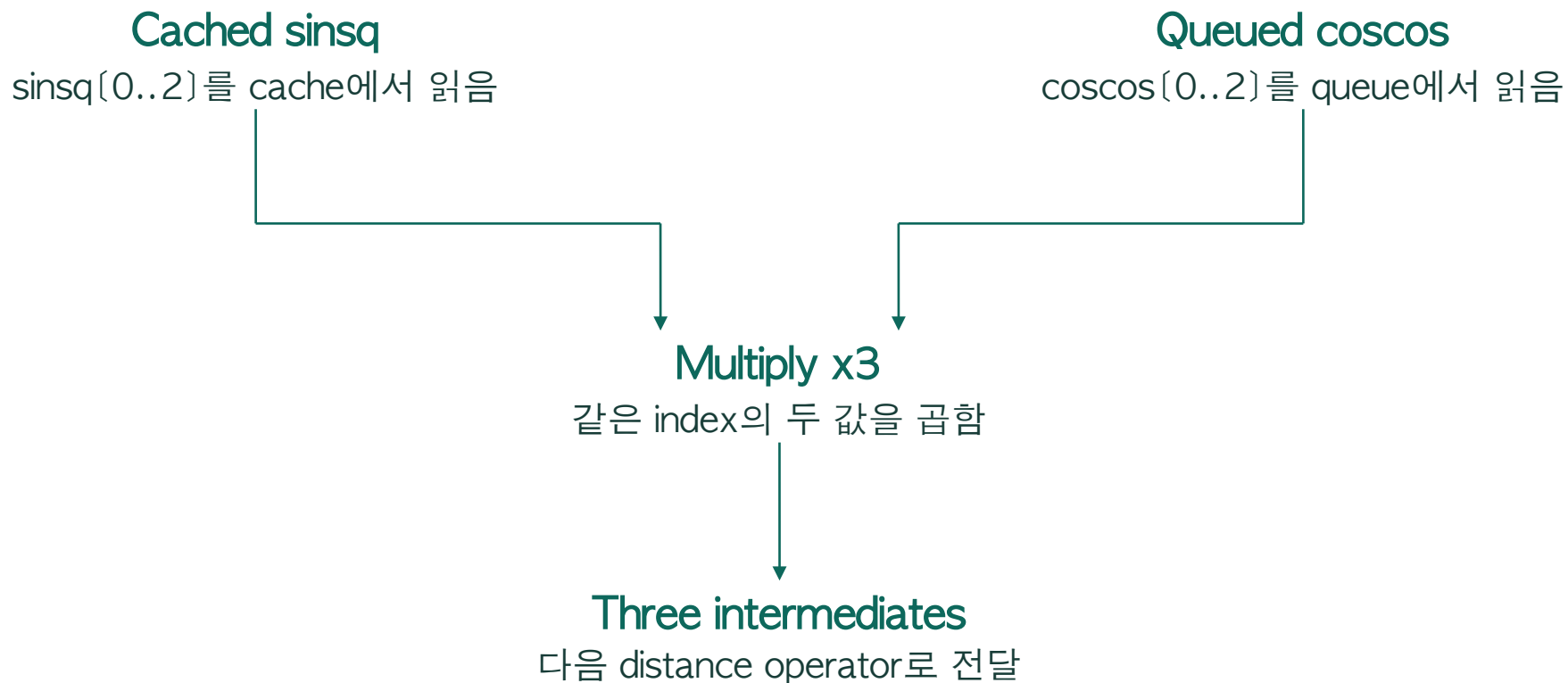
원문 Figure 2의 M-input block을 세 input으로 펼친 constructed example

01	Issue sinsq x3	Inputs 0, 1, 2의 request를 연속 생성
↓		
02	Stream results	Ready sinsq 값을 queue에서 cache로 이동
↓		
03	Issue coscos x3	동시에 다음 세 input을 operator에 공급
↓		
04	Wait for first ready	첫 coscos가 complete된 뒤 multiply 시작

M=3은 실행 순서를 설명하는 예시이며 measured cycle count나 speedup이 아니다.

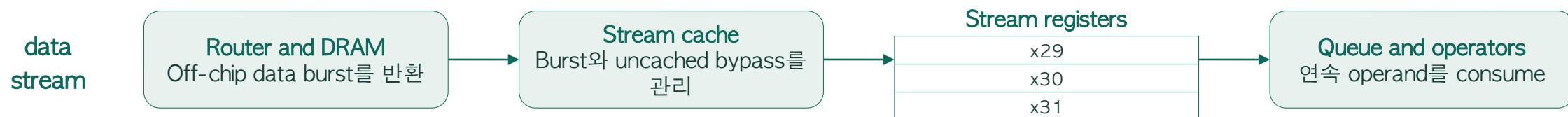
Multiply stage의 두 operand stream

M=3 constructed example의 같은 index 값을 결합한다



Stream-semantic memory path

Burst request가 word별 load/store instruction을 줄인다



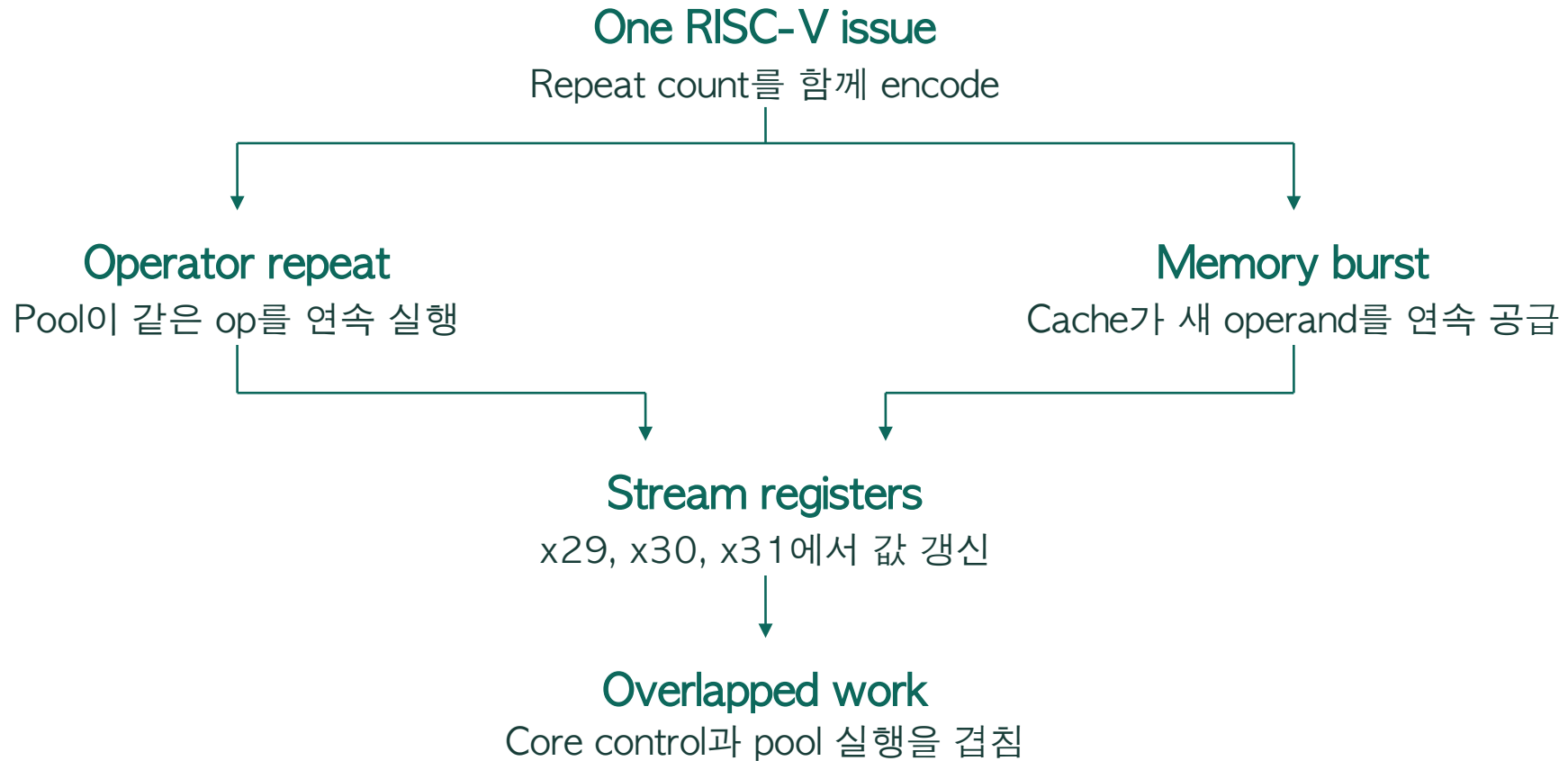
Stream cache는 normal RISC-V local memory와 분리되고 completion queue 전용으로 연결된다.

Memory request immediate의 12-bit 구성

Field	Bits	역할
Burst length	6	한 request가 다룰 stream 길이
Repeat count	5	같은 memory region의 반복 접근 횟수
Uncached flag	1	Cache 대신 BRAM bypass queue 사용

Burst와 repeat가 memory command 수를 줄이고 uncached bit가 bypass 여부를 정한다.

Operator repeat와 stream의 결합



Custom operator instruction encoding

R-type field	Labidus의 사용	주의점
Register fields	rd, rs1, rs2 register fields	Normal R-type 위치를 유지
func7	호출할 operator 선택	Generated design의 operator 집합
func3	Operator repeat count	Memory immediate의 repeat와 별도
6-bit opcode	한 instance에서 최대 64 operator types	Template의 수백 후보 중 사용분만 배정

Generated hardware마다 실제 사용 operator에 opcode를 다시 배정한다.

Shared operator pool과 trimming

01 Count operations Static analysis가 상대 frequency 추정



02 Share per tile Four cores가 one pool을 사용



03 Choose instances Operator마다 1개에서 4개 배치



04 Remove idle copies 낮은 utilization의 hardware를 줄임

Trimming은 사용하지 않는 operator와 과도한 core별 복제를 줄인다.

Workload별 4-core tile operator 구성

Workload	공통 arithmetic instances	Application-specific instances
Cholesky	multiply 2, divide 1, add 2, sqrt 1	none
FIR	multiply 2, add 2	none
FFT	multiply 2, add 3	none
K-means	multiply 2, add 2	sinsq 1, coscos 1, arcsrt 1

Operator pool은 workload마다 달라지는 hardware 구성이다.

평가 구현과 자원 조건

항목	Paper configuration	해석
FPGA platform	Amazon AWS F1, Xilinx Virtex UltraScale	Host-connected accelerator card
Synthesis clock	250 MHz	비교 normalization의 기준 clock
Operator-pool LUT	FFT 3.4% to Haversine 4.2%	Workload configuration에 따라 변화
Cache and processor	160 BRAM blocks, 7%	Four cores, caches, queues의 current setup

모든 resource 수치는 이 FPGA와 current tile configuration에 묶여 있다.

1024-slot latency-hiding resource 비교

Application runtime이 아니라 queue-style control hardware의 LUT estimate다

Design	조건	LUT result
Deep-pipeline proxy	1024-slot reorder buffer, matching ISA change 없음	More than 80K, about 7%
Labidus tile baseline	1024-entry completion queue, operator pool 제외	About 38K, about 3%

Paper는 이 비교를 2배가 넘는 resource-efficiency improvement로 해석한다.

네 application의 비교 조건

Application	Labidus condition	Compared implementations
Cholesky	Sub-matrix 256, matrix fits distributed on-chip memory	OmpSs, RTL, HLS, MicroBlaze
FIR	One independent stream per core	Xilinx, ANL HLS, soft GPU, MicroBlaze
FFT	256-point FFTs across cores	Xilinx, OpenCL, HLS, RTL, MBx32
K-means	Haversine comparisons across cores	Internal RTL and HLS, published RTL

Figure 6은 서로 다른 publication과 platform을 한 축에서 비교한다.

Cross-publication normalization의 순서

01 Collect published results Expert-optimized RTL과 HLS 결과 선택



02 Scale slower clocks Performance를 250 MHz로 선형 환산



03 Normalize LUT inputs FPGA family의 LUT input 수를 맞춤



04 Plot performance per LUT Given resource의 efficiency를 비교

Figure 6은 같은 board의 직접 대결이 아니라 normalization을 포함한 비교다.

성능 headline과 적용 범위

Paper statement

Over 50x vs MicroBlaze

Almost 10x vs soft GPUs

RTL is 4-7x faster

직접 조건

Chip resource당 performance headline

Published designs와 Cholesky-based normalization

Matrix multiplication과 N-body, similar resources

해석 경계

모든 workload의 raw runtime ratio가 아님

같은 board의 end-to-end measurement가 아님

Regular control kernel은 Labidus target이 아님

강점은 irregular control이며 regular dense kernel은 RTL이 앞설 수 있다.

온디바이스 hardware 판단에 주는 근거

Paper가 보여주는 것

FPGA 안에서 RV32I와 custom operator 결합

BRAM queue로 many in-flight operations 지원

Three stream registers와 burst memory path

Workload별 operator generation과 trimming

Paper만으로 보장할 수 없는 것

Smartphone 또는 MCU deployment

Battery와 thermal constraint 충족

Host 없이 standalone execution

Hard real-time, DNN, ASIC, NPU superiority

Labidus는 edge accelerator의 설계 재료를 제공하지만 mobile on-device 검증 논문은 아니다.

Labidus의 전체 연결

- | | | |
|----|---------------------|-------------------------------------|
| 01 | Analyze C kernel | 응용별 fused operator를 선택 |
| ↓ | | |
| 02 | Generate both paths | Hardware HDL과 modified software 생성 |
| ↓ | | |
| 03 | Feed streams | Burst와 three registers로 operand 공급 |
| ↓ | | |
| 04 | Hide latency | Completion queue가 requests를 겹침 |
| ↓ | | |
| 05 | Share and trim | Four cores의 operator utilization 개선 |

Labidus의 성능 논리는 compile-time specialization과 runtime latency hiding의 결합이다.